C++

BY: JANHAVI DEO

OOP Terminologies

Members variables / data members / fields: variables declared in the class

Member functions / methods: functions declared in the class

```
class date {
  int dd, mm, yy;

public:
  void set_date(int dd, int mm, int  yy)
   {
      …
      …
   }
  void display( )
   {
   }
};  //end of class
```

Member variables

Member function

Member function

We can declare as many member variables and member functions in a class.

Access Specifiers in C++

Private :

- We can access the private members from within the class only.
- We can not access the private members from outside the class.
- We keep data members as private, so that user of class can not modify the data from out side the class.
- We can declare some of the functions as private if needed. These functions will not be accessible from outside the class.

Access Specifiers in C++

Public :

- We keep member functions as public, because we want that the user of the class to be able to call functions using object of the class.
- Public means we can access the members from within the class as well from outside class.
- Public is used to provide interface of class. User of the class can talk with the object by calling public functions.

setter and getter methods

Data is kept private in the class. To access or modify the data members we write methods for all possible member variables. These methods are getters and setters. We can implement all possible data validations in these methods.

- **setter methods are used to set values of member variables.**

- **getter methods are used to get values of member variables.**

Setters and getters in date class

```
void setDay(int d)
 {
  if( d <= 31)  dd = d;
  else cout<<"\n Invalid day.. can not be set as value of dd...";
 }

 void setMonth(int m)
 {
  if( m <= 12)  mm = m;
  else cout<<"\n Invalid month.. can not be set as value of mm...";
 }

 void setYear(int y)
 {
  yy = y;
 }
```

```
int getDay()
 {
   return dd;
 }

 int getMonth()
 {
   return mm;
 }

 int getYear()
 {
   return yy;
 }
```

Assignment:

Add getters and setters in **time** and **point** class, and call these methods from main( ).

this pointer

- **this pointer** is passed implicitly in every function of class and it contains address of calling object.
- Each object gets its own copy of the data member inside member function through **this pointer.**
- The 'this' pointer is passed as a hidden argument to all member function calls and is available as a local variable within the body of all functions.
- Following are the situations where 'this' pointer is used:
  1) When local variable's name is same as member's name
  2) To return reference of the calling object from function.

Sample code to understand this Pointers

```
#include<iostream>
using namespace std;

class date {
  int dd, mm, yy;

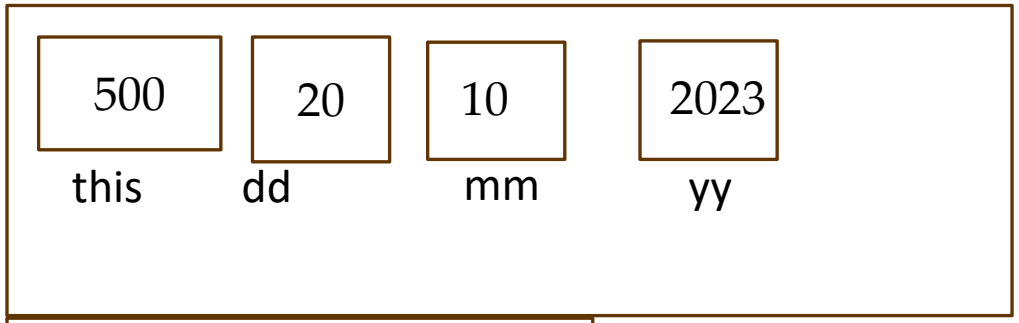
public:
  void set_date(int dd, int mm, int  yy)
   {
      this->dd = dd;
      this->mm = mm;
      this->yy = yy;
   }
  void display( )
   {
      cout << "\n"<< dd << "/" << mm <<" / "<<yy;
   }
};  //end of class
```

```
int main( )
{
    date obj1, obj2;
    obj1.set_date(20,10,2023);
    obj1.display( );
    obj2.set_date(23,10,2023);
    obj2.display( );
    return 0;
}
```

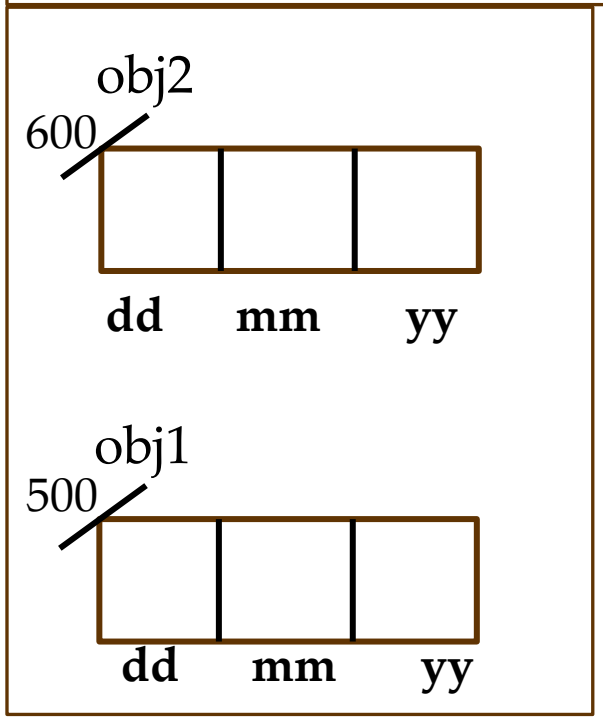
Stack frames in RAM on function call

obj1.set_date(20,10,2023);

stack frame of set_data( )



stack frame of main( )




```
void set_date(int dd, int mm, int  yy)
   {
      this->dd = dd;
      this->mm = mm;
      this->yy = yy;
   }
```